



МИНОБРНАУКИ РОССИИ

**федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)**

**Основная образовательная программа 09.03.02
«Информационные системы и технологии»**

**Отчет по дисциплине «Проектная деятельность»
по домашней работе на семинаре №2**

Тема: «Простой детектор без ML»

**Проверил
к.т.н. доцент**

Ковалев И.А.

подпись

**Выполнила
студентка группы ИДБ-25-06**

Якунина В.С.

подпись

Оглавление

ВВЕДЕНИЕ.....3

ПРОГРАММА.....3

ЗАКЛЮЧЕНИЕ.....7

ВВЕДЕНИЕ

Целью работы является знакомство с логикой анализа телеметрии космического аппарата и понимание ограничения правил, заданных человеком, до перехода к машинному обучению . Ссылка на [github](#) с программой.

ПРОГРАММА

Для выполнения задачи необходимо было доработать алгоритм, который анализирует данные о характеристиках спутника, анализирует и обрабатывает их. Детектор был сделан более гибким.

```
import pandas as pd
import matplotlib.pyplot as plt

def get_intervals(df, col='anomaly'):
    mask = df[col] == 1
    grp = (mask != mask.shift()).cumsum()

    intervals = (
        df[mask]
        .groupby(grp)
        .agg(
            start=('timestamp', 'first'),
            end=('timestamp', 'last'),
            rows=('timestamp', 'size')
        )
        .reset_index(drop=True)
    )
    intervals['duration_min'] = (
        (intervals['end'] - intervals['start']).dt.total_seconds() / 60
    )
    return intervals

def detect_combin(row):
    result = int(row['temperature'] > 65) + int(row['voltage'] < 23) +
    int(row['current'] > 9) + int(row['angular_velocity'] > 1.5)
    return int(result >= 2)
```

```

def detect_hard(row):
    return int(
        row['temperature'] > 65 or
        row['voltage'] < 23 or
        row['current'] > 9 or
        row['angular_velocity'] > 1.5
    )

def detect_stat(df):
    mean_t = df['temperature'].mean()
    std_t = df['temperature'].std()
    return (abs(df['temperature'] - mean_t) > 2 * std_t).astype(int)

def main():
    filename = "telemetry_dzz_sem1.csv"

    df = pd.read_csv(filename, parse_dates=['timestamp'])
    df['timestamp'] = pd.to_datetime(df['timestamp'])
    df = df.sort_values('timestamp').reset_index(drop=True)

    print('!!!!!!!!!!!!!!!!!!!! TELEMETRY (5) !!!!!!!!!!!!!!!!!!!!!')
    print(df.head())
    print()

    df['anomaly_hard'] = df.apply(detect_hard, axis=1)
    df['anomaly_stat'] = detect_stat(df)
    df['anomaly_comb'] = df.apply(detect_combin, axis=1)

    params = ['temperature', 'voltage', 'current', 'angular_velocity']
    for col in params:
        print("-----")
        print(col)
        print("-----")
        print(df.groupby('mode')[col].describe())
        print("-----")
    print()

    intervals_hard = get_intervals(df, 'anomaly_hard')
    intervals_stat = get_intervals(df, 'anomaly_stat')
    intervals_comb = get_intervals(df, 'anomaly_comb')

    print("*****")
    print("INTERVALS - hard")
    print("*****")
    print(intervals_hard)

```

```

print()
print("*****")
print("INTERVALS – statistical")
print("*****")
print(intervals_stat)
print()
print("*****")
print("INTERVALS – combined")
print("*****")
print(intervals_comb)
print()
print("*****")
print()

print("(((((((((((((((((((((")
print("DETECTOR COMPARISON")
print(f"Hard:          anomaly rows = {df['anomaly_hard'].sum()}, intervals = {len(intervals_hard)}")
print(f"Statistical:  anomaly rows = {df['anomaly_stat'].sum()}, intervals = {len(intervals_stat)}")
print(f"Combined:      anomaly rows = {df['anomaly_comb'].sum()}, intervals = {len(intervals_comb)}")
print("))))))))))))))))))")
print()

for col in params:
    plt.figure(figsize=(12, 4))
    plt.plot(df['timestamp'], df[col], label=col, color='tab:blue')

    plt.scatter(
        df.loc[df['anomaly_hard'] == 1, 'timestamp'],
        df.loc[df['anomaly_hard'] == 1, col],
        color='orange', s=25, label='hard', zorder=5
    )
    plt.scatter(
        df.loc[df['anomaly_stat'] == 1, 'timestamp'],
        df.loc[df['anomaly_stat'] == 1, col],
        color='red', s=30, label='stat', zorder=6
    )
    plt.scatter(
        df.loc[df['anomaly_comb'] == 1, 'timestamp'],
        df.loc[df['anomaly_comb'] == 1, col],
        color='purple', s=35, label='comb', zorder=7
    )

    for _, row in intervals_hard.iterrows():
        plt.axvspan(row['start'], row['end'], color='orange', alpha=0.10)

```

```

for _, row in intervals_stat.iterrows():
    plt.axvspan(row['start'], row['end'], color='red', alpha=0.10)
for _, row in intervals_comb.iterrows():
    plt.axvspan(row['start'], row['end'], color='purple', alpha=0.10)

plt.title(f'Telemetry: {col}')
plt.xlabel('timestamp')
plt.ylabel(col)

df['temperature_diff'] = df['temperature'].diff()
df['voltage_diff'] = df['voltage'].diff()
df['current_diff'] = df['current'].diff()
df['angular_velocity_diff'] = df['angular_velocity'].diff()

params = ['temperature_diff', 'voltage_diff', 'current_diff',
'angular_velocity_diff']
params_diff = [5, 0.4, 1, 0.1]
cnt = 0

for col in params:
    plt.figure(figsize=(12, 4))
    plt.plot(df['timestamp'], df[col], label=col, color='tab:blue')
    plt.scatter(
        df.loc[abs(df[col]) > params_diff[cnt], 'timestamp'],
        df.loc[abs(df[col]) > params_diff[cnt], col],
        color='green', s=25, zorder=5
    )
    plt.title(f'Telemetry: {col}')
    plt.xlabel('timestamp')
    plt.ylabel(col)
    cnt += 1

plt.show()
print()
print('xxxxxxxxxxxxxxxxxxxxx CLOSE FIGURES xxxxxxxxxxxxxxxxxxxxxxxx')
print()

if __name__ == "__main__":
    main()

```

ЗАКЛЮЧЕНИЕ

При выполнении задания был доработан простой детектор без машинного обучения. Ссылка на [github](#) с программой.